

# Full Stack Development with MERN Project Documentation format

## 1. Introduction

- **Project Title: Import Data Using Transform Maps (Spreadsheet)**

| Name        | Role                                  |
|-------------|---------------------------------------|
| Yugamathi.p | Individual participant / Project lead |

## 2. Project Overview

**Purpose:** To let users bulk-import spreadsheet data into a database without manual entry. Uploaded data is first held in a staging area. A configurable transform map then validates it and maps it to the target collection, with duplicate prevention.

**Features:**

- Upload .xlsx / .csv files
- Preview of the uploaded data before import
- Transform map builder (auto-map columns, edit mappings manually)
- Coalesce field to update existing records instead of duplicating them
- Data validation and cleaning (trim, type conversion, required fields)
- Import history with inserted, updated, and failed row counts
- Downloadable error log for failed rows
- Secure login with role-based access

## 3. Architecture

**Frontend (React)**

- Component-based UI built with React and React Router
- Main pages: Login, Upload, Transform Map Editor, Import History, Data Viewer
- Axios for API calls; Context API (or Redux) for state
- Reusable components: FileUploader, DataPreviewTable, MappingRow, ImportSummary

## Backend (Node.js and Express.js)

- REST API built with Express
- Multer handles file uploads; the `xlsx` library parses spreadsheets
- Service layer runs the transform engine: read staged rows → apply field maps → validate → insert or update
- Middleware for authentication, error handling, and validation Database

## (MongoDB with Mongoose)

| Collection                           | Key Fields                                                                   |
|--------------------------------------|------------------------------------------------------------------------------|
| <code>users</code>                   | name, email, password (hashed), role                                         |
| <code>importsets</code><br>(staging) | fileName, uploadedBy, rows[], status, createdAt                              |
| <code>transformmaps</code>           | name, targetCollection, fieldMappings[{source, target, type}], coalesceField |
| <code>importhistory</code>           | importSetId, mapId, inserted, updated, failed, errors[], runAt               |
| <code>records</code> (target)        | fields defined by the target schema, e.g. name, email, department            |

Flow: Spreadsheet → Upload API → Staging (`importsets`) → Transform Map → Target collection → Import History

## 4. Setup Instructions

### Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local install or MongoDB Atlas)
- Git
- A code editor (VS Code) and Postman (for API testing)

### Installation

#### Bash

# 1. Clone the repository

```
git clone [your-repo-url]
cd [project-folder]
```

# 2. Install server dependencies

```
cd server  
npm install
```

# 3. Install client dependencies

```
cd ../client  
npm install
```

Environment variables (create `server/.env`)

```
PORT=5000  
MONGO_URI=mongodb://localhost:27017/transformmap_db  
JWT_SECRET=your_secret_key  
JWT_EXPIRES_IN=1d
```

## 5. Folder Structure

Client

client

/

├── public/  
├── src/

├── components/ # FileUploader, DataPreviewTable, MappingRow  
├── pages/ # Login, Upload, MapEditor, History  
├── context/ # Auth and app state  
├── services/ # Axios API calls  
└── App.js

index.js

Server

server/

├── config/ # DB connection  
├── controllers/ # authController, importController, mapController  
├── models/ # User, ImportSet, TransformMap, ImportHistory, Record  
├── routes/ # authRoutes, importRoutes, mapRoutes  
├── middleware/ # authMiddleware, errorHandler, upload (Multipart)  
├── services/ # transformEngine.js, spreadsheetParser.js  
├── uploads/ # temporary file storage  
├── .env  
└── server.js

## 6. Running the Application

bash

# Backend (from the server  
directory) `npm start`

# Frontend (from the client directory)

`npm start`

- Frontend: <http://localhost:3000>
- Backend: <http://localhost:5000>

## 7. API Documentation

| Method | Endpoint                                | Description                   | Body / Params                                                     |
|--------|-----------------------------------------|-------------------------------|-------------------------------------------------------------------|
| POST   | <code>/api/auth/register</code>         | Register a user               | <code>name, email, password</code>                                |
| POST   | <code>/api/auth/login</code>            | Log in, receive token         | <code>email, password</code>                                      |
| POST   | <code>/api/imports/upload</code>        | Upload spreadsheet to staging | <code>file</code> (multipart)                                     |
| GET    | <code>/api/imports/:id</code>           | View staged rows              | <code>id</code> (param)                                           |
| POST   | <code>/api/maps</code>                  | Create a transform map        | <code>name, targetCollection, fieldMappings, coalesceField</code> |
| GET    | <code>/api/maps</code>                  | List transform maps           | none                                                              |
| POST   | <code>/api/imports/:id/transform</code> | Run the transform             | <code>mapId</code>                                                |
| GET    | <code>/api/history</code>               | View import history           | none                                                              |

Example: `POST /api/imports/:id/transform`

json

// Request

```
{ "mapId": "66f1a2b3c4d5e6f7a8b9c0d1" }
```

// Response (200)

```
{
  "message": "Transform completed", "inserted": 95,
  "updated": 3,
  "failed": 2,
  "errors": [{ "row": 14, "reason": "Email is required" }]
}
```

## 8. Authentication

- Users register and log in with email and password; passwords are hashed with bcrypt.
- On login, the server issues a JWT signed with `JWT_SECRET`.
- The client stores the token and sends it in the `Authorization: Bearer <token>` header.
- An auth middleware verifies the token on protected routes.
- Authorization: roles (e.g., `admin`, `user`) restrict actions; only admins can create transform maps and run imports.

## **9. User Interface**

Add screenshots or GIFs of:

1. Login / Register page
2. File upload page with data preview
3. Transform map editor (source → target mapping)
4. Import result summary (inserted / updated / failed)
5. Import history page
6. Target data viewer

## **10. Testing**

- **Unit testing:** Jest for the transform engine and parser functions
- **API testing:** Postman collections, plus Supertest for automated endpoint tests
- **Frontend testing:** React Testing Library for components
- **Manual testing:** valid files, invalid formats, empty files, duplicate rows, missing required fields
- **Performance check:** import time for [N] rows

## **11. Screenshots or Demo**

- **Screenshots:** [Insert images]
- **Demo video link:** [Insert link]
- **GitHub repository:** [Insert link]

## **12. Known Issues**

- **Very large files (over [X] MB) may be slow or hit upload limits**

- Only .xlsx and .csv formats are supported
- Merged cells or multiple header rows in a spreadsheet can cause parsing errors
- Date formats vary across spreadsheets and may need manual mapping
- No rollback of an import once it is completed

## 4. Setup Instructions

### Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local install or MongoDB Atlas)
- Git
- A code editor (VS Code) and Postman (for API testing)

### Installation

bash

# 1. Clone the repository

## 13. Future Enhancements

- Scheduled and automated imports
- Undo/rollback of a completed import
- Support for more sources (Google Sheets, REST APIs, databases)
- Custom transform scripts for advanced data cleaning
- Email notifications on import completion or failure
- Background processing with a job queue for large files
- Analytics dashboard for import success rates